Software Requirements Specification (SRS)

Project: Healthcare Appointment and Patient Record Manager

Version: 1.0 (initial draft)

Authors:

KARANAM SREEHITHA – PES1UG24AM129

KHUSHI PATEL – PES1UG24AM134

KIRTHANA S -- PES1UG24AM136

PORLA NEHA -- PES1UG25AM808

Status: Reviewed by Project Team

Revision History

Version	Date	Author	Change Summary	Approval
1.0	28-10-2026	Project team	Initial project-specific draft adapted from provided template	Approved by Project Team

Approvals

Role	Name	Signature / Email	Date
Team Lead	KARANAM SREEHITHA	sreehithakaranam@gamil.com	28-10-2026
Team Member	KHUSHI HIRENKUMAR PATEL KIRTHANA S PORLA NEHA	khushihpatel06@gmail.com Kirthanashetty17@gmail.com porlaneha1@gmail.com	28-10-2026
Course TA / Evaluator			

Repository: https://github.com/Khushi-patel05/Team4_SE_Mini_Project

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for the Healthcare Appointment and Patient Record Manager. It provides a shared baseline for implementation, testing, sprint planning and review.

1.2 Scope
The application is a web-based healthcare administration prototype built with Python Flask and SQLite. The verified current implementation supports patient registration, patient search/view/edit/delete, appointment booking/listing/filtering, appointment editing and appointment status updates. Medical history, prescription management, user authentication and dashboards are not confirmed as implemented in the current verified baseline and are treated as planned requirements until demonstrated and tested.

1.3 Audience
Student developers, the team lead, testers, the course TA/instructor and project evaluators.

1.4 Definitions
FR: Functional Requirement; NFR: Non-Functional Requirement; SRS: Software Requirements Specification; SAD: Software Architecture and Design; STP: Software Test Plan; RTM: Requirements Traceability Matrix; CRUD: Create, Read, Update, Delete; UI: User Interface; API: Application Programming Interface.

2. Overall description

2.1 Product perspective
The system is a small web application with browser-based HTML templates, Flask request handling and a SQLite database. It is intended for a coursework prototype and is not a production clinical-record system.

2.2 Major product functions
- Register and maintain patient demographic/contact records.
- Search for patients and view a patient's details.
- Edit and delete patient records, subject to validation and confirmation safeguards.
- Book appointments for registered patients.
- View appointments and filter the list using supported fields/status.
- Edit appointment details and update appointment status.
- Validate required fields and report invalid input to the user.
- Provide a health-check endpoint for basic application availability.
- Planned, pending verification: role-based login, medical history, prescription management, dashboard/reporting and audit history.

2.3 User roles and characteristics

Role	Description	Access status
Clinic staff / receptionist	Enters patient details and schedules appointments.	Core workflow is the current intended user workflow; authentication not verified.
Administrator	Would manage access and oversee records.	Planned; role-based authentication not verified.
Developer / tester	Runs the app, maintains code and validates behavior.	Development/test role.

2.4 Operating environment
Development and demonstration environment: Python 3.11-compatible virtual environment, Flask, SQLite, a modern web browser, and a desktop OS such as Windows. Exact deployment platform and production hosting are not specified.

2.5 Constraints
- The current implementation uses a local SQLite database; concurrent multi-user production deployment has not been established.
- Only features confirmed in the current code and tests should be represented as implemented.
- Real patient information must not be used in coursework demonstrations; use synthetic test data.
- Authentication, authorization, encryption at rest, backups and regulatory compliance are not verified for this prototype.

3. External interfaces

3.1 User interfaces

The browser UI is rendered using Flask templates and static CSS. Pages cover patient listing/registration, patient detail/editing, and appointment listing/booking/editing. UI labels and validation feedback should be clear and consistent.

3.2 Hardware interfaces

No dedicated hardware interface is required or confirmed.

3.3 Software interfaces

- Python Flask application framework.
- SQLite database accessed by the application.
- HTML templates and CSS served to a web browser.
- Automated tests executed with pytest; coverage can be measured with coverage.py/pytest-cov if configured.

3.4 Communications

The browser communicates with the Flask server over HTTP in a local development environment. HTTPS/TLS configuration for deployment is not verified and must be added before any real or sensitive data is handled.

Security objective: protect patient information from unauthorized access and accidental exposure. Security objective: maintain accurate and consistent patient and appointment records.

4. System features

Priority values: High = core project workflow; Medium = supporting or planned capability.

Status is based on the currently verified implementation snapshot and must be updated after team review.

Req ID	Requirement	Type	Priority	Source/module	Acceptance criteria	Status
FR-001	The system shall allow staff to register a patient using required patient fields.	Functional	High	Patient module	Valid patient is saved and appears in the patient list.	Implemented / test coverage reported
FR-002	The system shall validate patient input and reject missing or invalid required values.	Functional	High	Patient module	Invalid input produces a validation response and is not saved.	Implemented / test coverage reported
FR-003	The system shall display a list of registered patients.	Functional	High	Patient module	Existing records are displayed in the patient list.	Implemented / test coverage reported
FR-004	The system shall allow staff to search patients using supported search fields.	Functional	High	Patient module	Matching patients are returned; non-matches are handled clearly.	Implemented / test coverage reported
FR-005	The system shall display the details of a selected patient.	Functional	High	Patient module	The selected patient's stored fields are shown.	Implemented / test coverage reported
FR-006	The system shall allow staff to edit an existing patient record.	Functional	High	Patient module	Valid edits persist; invalid edits are rejected.	Implemented / test coverage reported
FR-007	The system shall allow staff to delete a patient record according to application rules.	Functional	Medium	Patient module	Deletion outcome is consistent and relatedrecord behavior is handled.	Implemented / test coverage reported; confirm FK behavior
FR-008	The system shall allow staff to book an appointment for a patient.	Functional	High	Appointment module	Valid appointment is stored and visible in the list.	Implemented / test coverage reported
FR-009	The system shall validate appointment input before saving.	Functional	High	Appointment module	Invalid appointment data is rejected with feedback.	Implemented / test coverage reported
FR-010	The system shall list scheduled appointments.	Functional	High	Appointment module	Stored appointments are displayed.	Implemented / test coverage reported
FR-011	The system shall filter appointments using supported filter criteria.	Functional	Medium	Appointment module	Only appointments matching the selected criteria are returned.	Implemented / test coverage reported
FR-012	The system shall allow staff to edit an existing appointment.	Functional	High	Appointment module	Valid changes persist and are shown on subsequent retrieval.	Implemented / test coverage reported

FR-013	The system shall allow staff to update an appointment's status.	Functional	High	Appointment module	A supported status change persists and is displayed.	Implemented / test coverage reported
FR-014	The system shall expose a basic healthcheck endpoint.	Functional	Medium	Application	Health endpoint returns the expected success response when running.	Implemented / test coverage reported
FR-015	The system shall show	Functional	High	Routes/templates	Error paths return a	Implemented / test
	understandable feedback when a requested record is missing or input is invalid.				controlled response rather than an unhandled exception.	coverage reported
FR-016	The system should authenticate staff before allowing access to patient records.	Functional / Security	High	Authentication	Unauthenticated requests cannot access protected pages.	Planned / not verified
FR-017	The system should support medical-history records linked to a patient.	Functional	Medium	Medical history	History entries can be created, viewed and retrieved for the correct patient.	Planned / not verified
FR-018	The system should support prescription records linked to a patient and, where applicable, an appointment.	Functional	Medium	Prescription module	Prescription data is stored and associated with the correct record.	Planned / not verified
FR-019	The system should provide a dashboard with useful patient/appointment summaries.	Functional	Low	Dashboard	Summary counts agree with stored records.	Planned / not verified

5. Non-functional requirements

Req ID	Requirement	Category	Priority	Acceptance criteria / measurement
NFR-001	The system shall complete ordinary local CRUD requests within 3 seconds under a small test dataset and normal local load.	Performance	Medium	Measure response times across a defined sample; report results rather than assume compliance.
NFR-002	The system shall preserve valid patient and appointment changes in SQLite after successful operations.	Reliability / integrity	High	Create or update a record, retrieve it again, and compare stored values.
NFR-003	The system shall validate user input on the server before database writes.	Security / robustness	High	Invalid and boundary-value inputs do not create or corrupt records.
NFR-004	The UI shall provide readable labels, clear validation messages and consistent navigation.	Usability	Medium	Representative users can complete patient and appointment tasks without unexplained errors.
NFR-005	The application shall be maintainable through modular routes, configuration, templates and automated tests.	Maintainability	High	Tests can be run from documented setup instructions; changes do not break existing test cases.
NFR-006	The system shall not expose passwords, secrets or sensitive patient data in error messages or logs.	Security / privacy	High	Inspect logs and error responses using synthetic test data.

5.1 Security

5.1.1 Security Objectives

- SO-1: Confidentiality — restrict patient and appointment information to authorized users before real-world use.
- SO-2: Integrity — prevent invalid or unintended changes to patient and appointment records and preserve database consistency.

5.1.2 Security Requirements

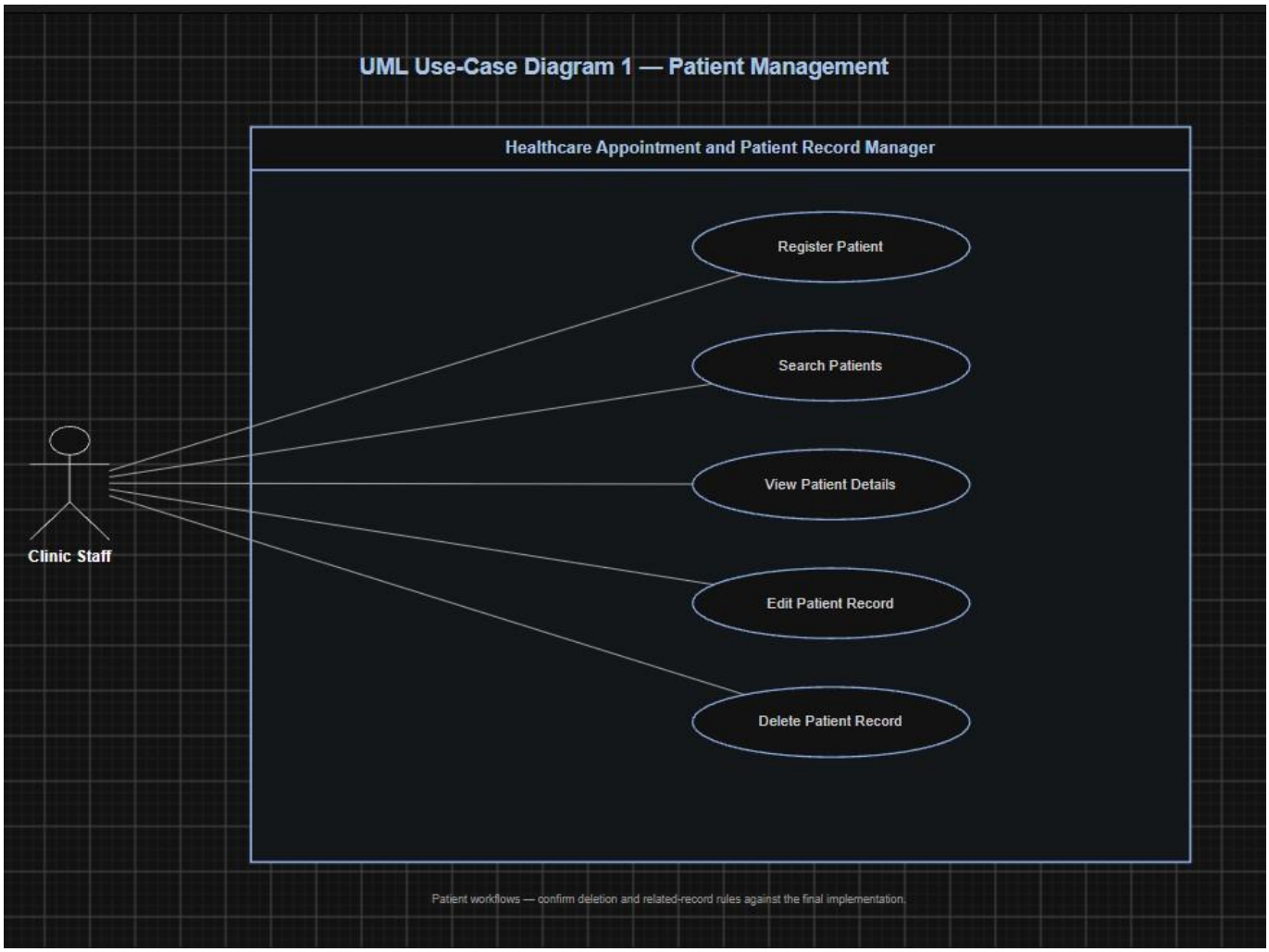
Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / test
SR-001	The deployed system shall use HTTPS/TLS for browserto-server communication.	Security	High	Verify HTTPS configuration in the deployment environment; currently not verified.
SR-002	The system shall authenticate users before exposing patient or appointment data.	Access control	High	Unauthenticated access is denied; currently planned/not verified.
SR-003	The system shall authorize each protected operation according to the user's role.	Access control	High	Unauthorized operations are denied; currently planned/not verified.
SR-004	The system shall validate and safely handle all user-	Input validation	High	Invalid inputs do not result in unexpected writes or
	supplied input before database operations.			unhandled errors.
SR-005	The system shall use synthetic data in development/testing and shall not place real patient records in public repositories.	Privacy	High	Repository and test fixtures contain no real patient data or credentials.

6. Quality attributes & Acceptance tests

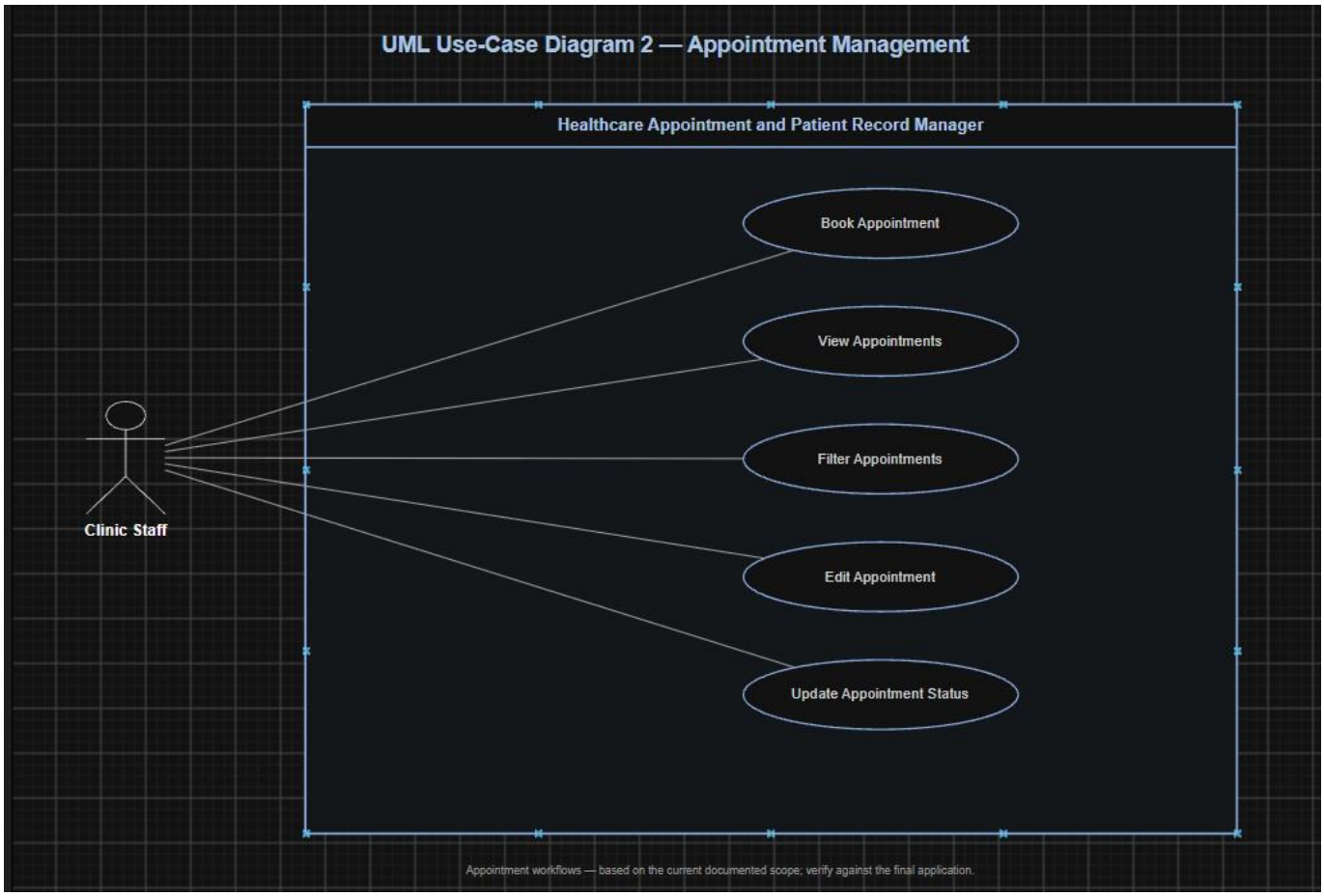
Quality attribute	Acceptance test / evidence
Functional correctness	Run patient and appointment automated tests; verify CRUD and validation behavior.
Data integrity	Create, update and retrieve records; verify values and patientappointment links.
Usability	Walk through patient registration, search, details and appointment booking with a reviewer.
Performance	Measure representative local requests with a defined test dataset; record environment and results.
Security and privacy	Verify input validation, access control if implemented, safe error handling, and absence of secrets/real patient data.
Maintainability	Run the documented test suite and CI checks from a clean checkout.

7. UML Use-Case Diagram

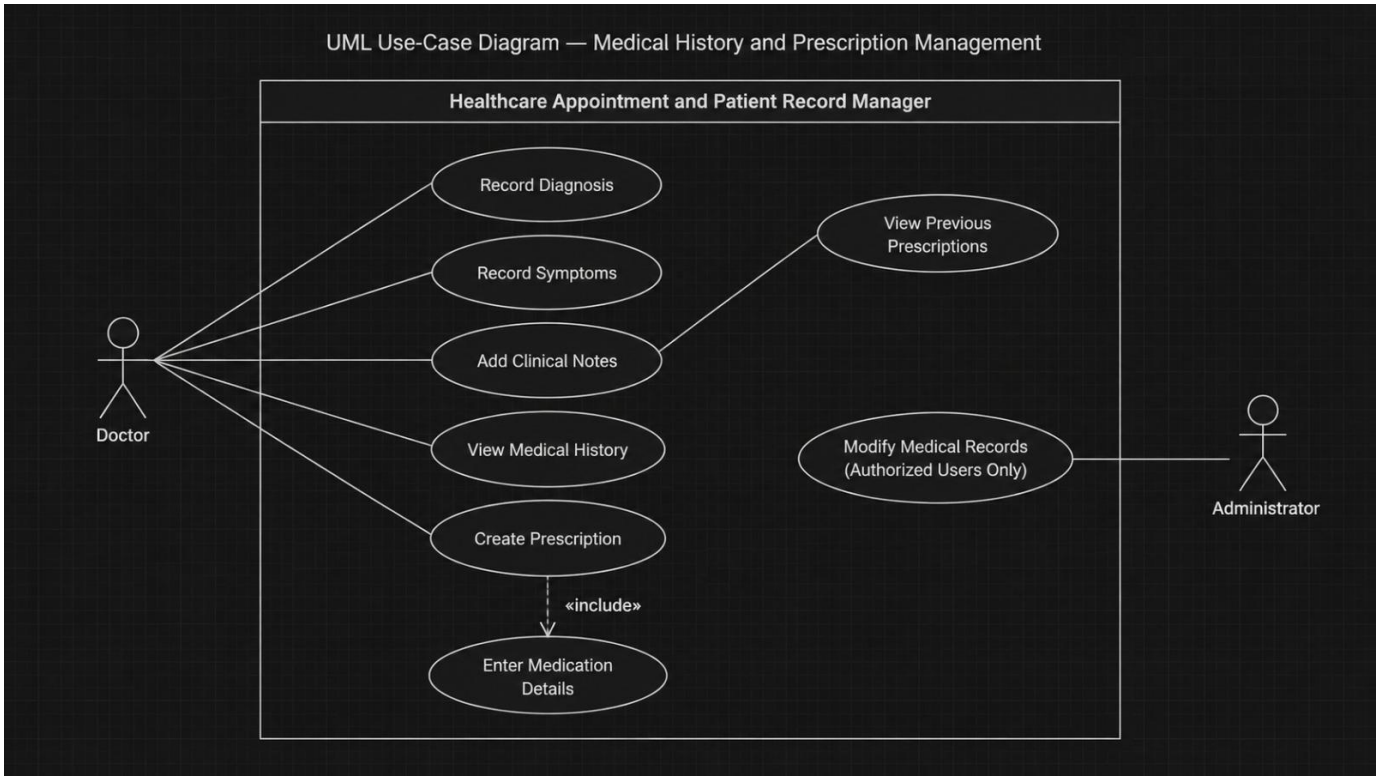
1.


UML Use-Case Diagram 1 — Patient Management
Healthcare Appointment and Patient Record Manager
Register Patient
Search Patients
View Patient Details
Edit Patient Record
Delete Patient Record
Clinic Staff
Patient workflows — confirm deletion and related-record rules against the final implementation.

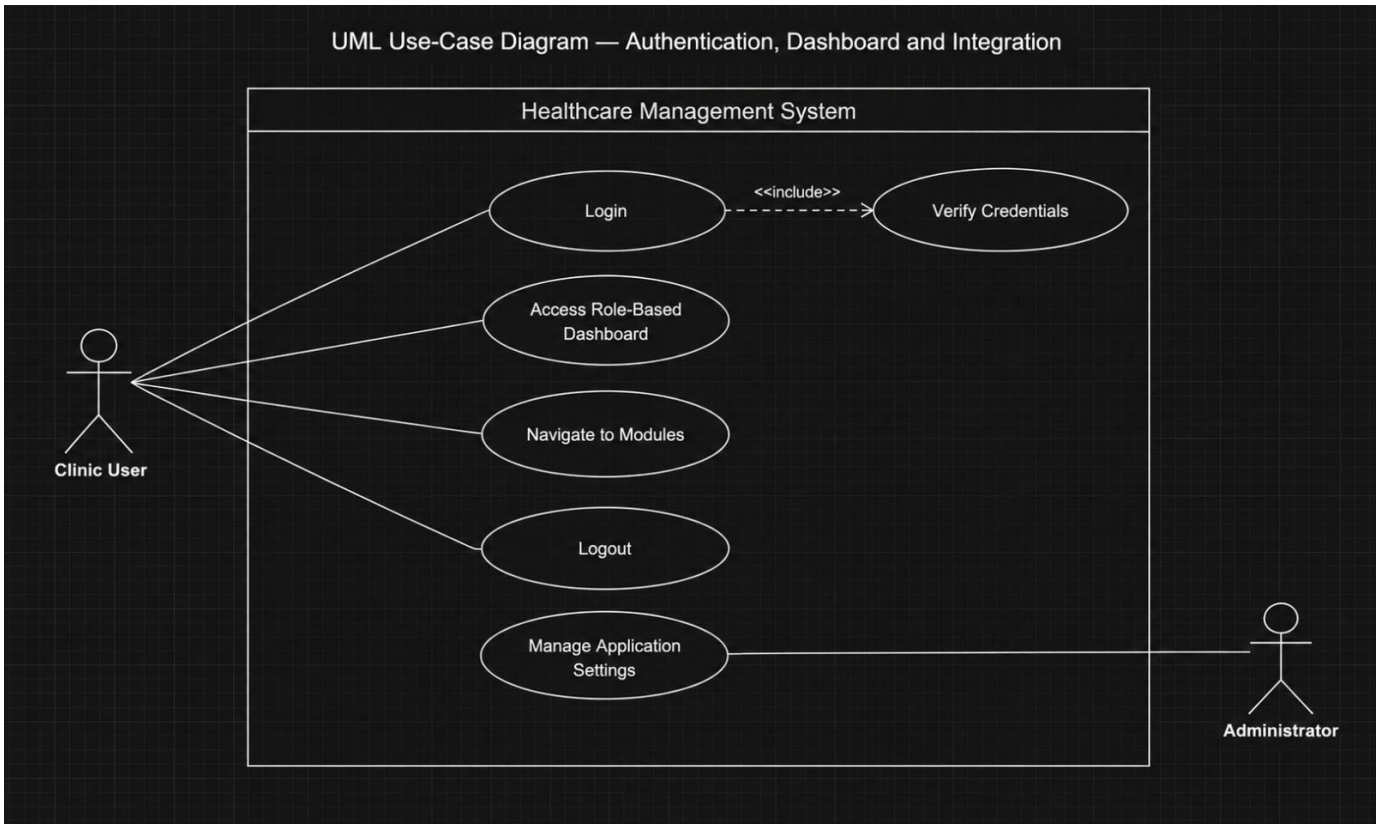
2.


UML Use-Case Diagram 2 — Appointment Management
Healthcare Appointment and Patient Record Manager
Book Appointment
View Appointments
Filter Appointments
Edit Appointment
Update Appointment Status
Clinic Staff
Appointment workflows — based on the current documented scope; verify against the final application.

3.


UML Use-Case Diagram — Medical History and Prescription Management
Healthcare Appointment and Patient Record Manager
Record Diagnosis
Record Symptoms
Add Clinical Notes
View Medical History
Create Prescription
«include»
Enter Medication Details
View Previous Prescriptions
Modify Medical Records (Authorized Users Only)
Doctor
Administrator

4.



8. Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section ref / design	Module	Test case(s)	Status	Comments
FR-001	allow staff to register a patient using required patient fields.	4 / Patient/Appointment routes	Patient module	TC-FR001	Implemented	Implemented / test coverage reported
FR-002	validate patient input and reject missing or invalid required val	4 / Patient/Appointment routes	Patient module	TC-FR002	Implemented	Implemented / test coverage reported
FR-003	display a list of registered patients.	4 / Patient/Appointment routes	Patient module	TC-FR003	Implemented	Implemented / test coverage reported
FR-004	allow staff to search patients using supported search fields.	4 / Patient/Appointment routes	Patient module	TC-FR004	Implemented	Implemented / test coverage reported
FR-005	display the details of a selected patient.	4 / Patient/Appointment routes	Patient module	TC-FR005	Implemented	Implemented / test coverage reported
FR-006	allow staff to edit an existing patient record.	4 / Patient/Appointment routes	Patient module	TC-FR006	Implemented	Implemented / test coverage reported
FR-007	allow staff to delete a patient record according to application r	4 / Patient/Appointment routes	Patient module	TC-FR007	Implemented	Implemented / test coverage reported; confirm FK behavior
FR-008	allow staff to book an appointment for a patient.	4 / Patient/Appointment routes	Appointment module	TC-FR008	Implemented	Implemented / test coverage reported
FR-009	validate appointment input before saving.	4 / Patient/Appointment routes	Appointment module	TC-FR009	Implemented	Implemented / test coverage reported
FR-010	list scheduled appointments.	4 / Patient/Appointment routes	Appointment module	TC-FR010	Implemented	Implemented / test coverage reported
FR-011	filter appointments using supported filter criteria.	4 / Patient/Appointment routes	Appointment module	TC-FR011	Implemented	Implemented / test coverage reported
FR-012	allow staff to edit an existing appointment.	4 / Patient/Appointment routes	Appointment module	TC-FR012	Implemented	Implemented / test coverage reported
FR-013	allow staff to update an appointment's status.	4 / Patient/Appointment routes	Appointment module	TC-FR013	Implemented	Implemented / test coverage reported
FR-014	expose a basic healthcheck endpoint.	4 / Application	Application	TC-FR014	Implemented	Implemented / test coverage reported
FR-015	show understandable feedback when a requested record is missing o	4 / Routes/templates	Routes/templates	TC-FR015	Implemented	Implemented / test coverage reported
FR-016	authenticate staff before allowing access to patient records.	4 / Authentication	Authentication	TC-FR016	Planned	Planned / not verified
FR-017	support medical-history records linked to a patient.	4 / Medical history	Medical history	TC-FR017	Planned	Planned / not verified
FR-018	support prescription records linked to a patient and, where appli	4 / Prescription module	Prescription module	TC-FR018	Planned	Planned / not verified
FR-019	provide a dashboard with useful patient/appointment	4 / Dashboard	Dashboard	TC-FR019	Planned	Planned / not verified

	summaries.					
NFR-001	The system shall complete ordinary local CRUD requests within 3 s	5 / Quality attributes	Application-wide	TC-NFR001	To verify	Measure response times across a defined sample; report results rather than assume compliance.
NFR-002	The system shall preserve valid patient and appointment changes i	5 / Quality attributes	Application-wide	TC-NFR002	To verify	Create or update a record, retrieve it again, and compare stored values.
NFR-003	The system shall validate user input on the server before databas	5 / Quality attributes	Application-wide	TC-NFR003	To verify	Invalid and boundary-value inputs do not create or corrupt records.
NFR-004	The UI shall provide readable labels, clear validation messages a	5 / Quality attributes	Application-wide	TC-NFR004	To verify	Representative users can complete patient and appointment tasks without unexplained errors.
NFR-005	The application shall be maintainable through modular routes, con	5 / Quality attributes	Application-wide	TC-NFR005	To verify	Tests can be run from documented setup instructions; changes do not break existing test cases.
NFR-006	The system shall not expose passwords, secrets or sensitive patie	5 / Quality attributes	Application-wide	TC-NFR006	To verify	Inspect logs and error responses using synthetic test data.

Team

Team member	USN
KARANAM SREEHITHA	PES1UG24AM129
KHUSHI HIRENKUMAR PATEL	PES1UG24AM134
KIRTHANA S	PES1UG24AM136
PORLA NEHA	PES1UG25AM808